

How to Install OpenClaw Locally — The Complete Guide

This guide walks you through setting up a 24/7 AI agent on **your own computer** inside a Docker container. It runs with a built-in browser, video tools, and a Telegram bot so you can talk to it from your phone. Your machine is locked down with a firewall — only the ports OpenClaw needs are open, and only on localhost.

Written for people with zero coding experience. Every word, every command, every concept is explained.

Security model: UFW firewall + Docker isolation + localhost-only ports = your agent is sealed tight. Nothing is exposed to the internet.

What You'll Have When You're Done

- An AI agent running 24/7 on your own computer (as long as it's on)
- A Telegram bot on your phone that lets you talk to the agent
- A built-in browser so the agent can log into websites and take actions for you
- Video tools (FFMPEG) for creating and editing videos
- Everything runs inside ONE sealed Docker container
- A firewall that blocks all incoming connections
- Auto-restarts so the container comes back after reboots

Before You Start — What You Need

- A computer running **Linux** (Ubuntu, Debian, Arch, Fedora, etc.) or **macOS**
- A Telegram account (free)
- A Claude Max or Claude Pro subscription, OR an Anthropic API key
- About 1–2 hours

What It Costs

Service	Cost
Your own computer	Free (you already have it)
Docker	Free
Telegram Bot API	Free
Claude AI (Max plan or API)	\$20–200/month depending on plan

Important Words You'll See in This Guide

Word	What It Means
Docker	A sealed box (called a "container") that runs programs inside it. If something goes wrong inside the box, your computer stays safe.
Container	The sealed box Docker creates. Your AI agent lives inside this box.
Terminal	The app on your computer where you type commands. It's a black/dark window with text.
UFW	A firewall — it blocks unwanted visitors from reaching your computer. Like a bouncer at a club.
Firewall	A digital wall that blocks people from connecting to your computer. Only lets approved traffic through.
localhost / 127.0.0.1	Your computer talking to itself. When a port is bound to localhost, only YOUR computer can access it — nobody else on the network can.
Port	Like a door number on a building. Different programs use different ports. OpenClaw uses port 18789.
sudo	"Super User DO" — lets you run one admin command at a time.
nano	A simple text editor inside the terminal. Like Notepad but for the terminal.
Token	A long secret code (like a super-password) that proves you have permission to use something.

PHASE 1: Install Docker

Docker is the sealed box your AI agent will live in. We install it first.

Step 1.1: Install Docker

Ubuntu / Debian:

```
sudo apt update
```

```
sudo apt install -y docker.io
```

Arch / CachyOS:

```
sudo pacman -S docker
```

Fedora:

```
sudo dnf install -y docker
```

macOS:

Download and install **Docker Desktop** from <https://www.docker.com/products/docker-desktop/>

Open Docker Desktop after installing. It needs to be running whenever you want your agent to run.

Step 1.2: Make Docker Start Automatically

Linux only (macOS Docker Desktop handles this automatically):

```
sudo systemctl enable --now docker
```

This tells your computer: "Always start Docker when the computer turns on."

Step 1.3: Let Your User Run Docker

Linux only:

```
sudo usermod -aG docker $USER
```

This lets you control Docker without typing `sudo` every time.

IMPORTANT: You need to log out and log back in for this to take effect. Or run:

```
newgrp docker
```

Step 1.4: Install Docker Compose

Ubuntu / Debian:

```
sudo apt install -y docker-compose-v2
```

Arch / CachyOS:

```
sudo pacman -S docker-compose
```

Fedora:

```
sudo dnf install -y docker-compose-plugin
```

macOS:

Docker Compose comes included with Docker Desktop. You're already set.

Step 1.5: Verify Docker Works

Run these 3 commands to make sure everything is installed:

```
docker --version
```

Should show something like: `Docker version 28.x.x`

```
docker compose version
```

Should show something like: `Docker Compose version 2.x.x`

```
docker run hello-world
```

Should print "Hello from Docker!" If you see this, Docker is working.

PHASE 2: Set Up the Firewall

We're going to lock down your computer so no incoming connections are allowed. Your agent only needs to make OUTGOING connections (to talk to APIs, Telegram, etc.). Nobody needs to connect IN to your machine.

Step 2.1: Install UFW

Ubuntu / Debian:

```
sudo apt install -y ufw
```

If it says "ufw is already the newest version" — that's fine, it was pre-installed.

Arch / CachyOS:

```
sudo pacman -S ufw
```

Fedora:

Fedora uses `firewalld` by default, which works similarly. You can either use `firewalld` or install `ufw`:

```
sudo dnf install -y ufw
```

macOS:

macOS has a built-in firewall. Go to **System Settings** → **Network** → **Firewall** and turn it **ON**. Then set it to "Block all incoming connections" (you can allow specific apps if needed). Skip to Phase 3 — the macOS firewall handles this.

Step 2.2: Set Firewall Rules

Run these commands **one at a time**:

```
sudo ufw default deny incoming
```

You should see: "Default incoming policy changed to 'deny'" — Good!

```
sudo ufw default allow outgoing
```

You should see: "Default outgoing policy changed to 'allow'" — Good!

Step 2.3: Enable the Firewall

```
sudo ufw enable
```

It asks: "Command may disrupt existing ssh connections. Proceed?" → Type **y**

You should see: "Firewall is active and enabled on system startup" — Good!

What did we just do?

- **Deny all incoming** = Nobody from the network can connect to your computer's services
- **Allow all outgoing** = Your computer CAN still reach the internet (it needs to download stuff, talk to APIs, reach Telegram, etc.)

Your agent can talk OUT to the internet, but nobody can reach IN to your machine.

Step 2.4: Verify the Firewall

```
sudo ufw status verbose
```

You should see:

```
Status: active
```

```
Default: deny (incoming), allow (outgoing), disabled (routed)
```

This means: "Block everything incoming. Allow everything outgoing."

"But wait — don't I need to open ports for OpenClaw?"

No! Here's why: OpenClaw's ports (18789, 18790, 6080, 9222) are all bound to **localhost (127.0.0.1)** inside the Docker container. Localhost traffic **bypasses the firewall entirely** — it never touches the network. It's your computer talking to itself. The firewall only blocks traffic coming from OTHER computers. So your agent works perfectly, and the firewall blocks everyone else.

PHASE 3: Install OpenClaw

This is the big one. We're going to install OpenClaw with ALL tools baked into a SINGLE container: the AI brain, the browser (Chromium), video tools (FFMPEG), and a screen viewer (NoVNC) so you can see the browser.

Step 3.1: Create the Workspace

```
mkdir -p ~/video-workspace/{clips,assets,output,projects,fonts,templates}
```

This creates folders where your agent will store videos and project files.

Step 3.2: Download OpenClaw

```
git clone https://github.com/openclaw/openclaw.git ~/openclaw-setup
```

```
cd ~/openclaw-setup
```

Step 3.3: Build the Docker Image WITH All Tools

This is where we bake everything into ONE container from the start. No separate containers, no networking headaches.

```
export OPENCLAW_DOCKER_APT_PACKAGES="ffmpeg chromium xvfb x11vnc novnc fonts-liberation
export OPENCLAW_EXTRA_MOUNTS="$HOME/video-workspace:/home/node/video-workspace"
./docker-setup.sh
```

What each package is:

Package	What It Does
ffmpeg	Video editing tool — cuts, joins, converts videos
chromium	Web browser — lets the agent browse websites, log into your accounts
xvfb	Virtual display — creates a fake "screen" inside the container so Chromium has somewhere to render
x11vnc	Screen sharing server — lets you view the virtual screen
novnc	Web-based screen viewer — lets you see the browser in YOUR web browser
fonts-liberation	Common fonts so websites look normal
fonts-noto-color-emoji	Emoji support

You might see a **"DEPRECATED: legacy builder" warning**. Ignore it. It's just a warning, not an error. The build will take several minutes — be patient.

Step 3.4: The Onboarding Wizard

After the build finishes, a setup wizard will appear. Here are the correct answers:

Question	What to Pick
"I understand this is powerful and inherently risky. Continue?"	Type <code>y</code>

Question	What to Pick
Local gateway or remote gateway?	Local gateway
Workspace directory?	Just press Enter to accept the default
Gateway bind?	Loopback (127.0.0.1) — this means only YOUR computer can access OpenClaw
Gateway auth?	Token — requires a secret code to access
Gateway token?	Press Enter to auto-generate one
DM access policies?	Pairing (this is the default — keep it)
Configure skills?	Skip for now
Preferred node manager?	Pick npm (or whatever you prefer)
Shell completion?	Your choice, doesn't matter

CRITICAL: When it generates a gateway token, **SAVE IT**. Write it down or copy it somewhere safe. You need this token to access the web dashboard later.

Item	Your Value
Gateway Token	_____

PHASE 4: Consolidate to One Container

Right now, the `docker-setup.sh` created multiple services. We're going to merge everything into ONE single container so there are no networking issues between containers.

Step 4.1: Create the Unified Startup Script

If you use **Kitty terminal**, run this first or you'll get an error:

```
export TERM=xterm
```

Now create the startup script:

```
nano ~/openclaw-setup/unified-entrypoint.sh
```

Type (or paste) this exactly:

```
#!/bin/bash
set -e

# Start virtual display (fake screen for the browser)
Xvfb :99 -screen 0 1280x720x24 -ac &
sleep 1
export DISPLAY=:99

# Start Chromium browser for remote debugging
chromium --no-sandbox --disable-gpu --headless=new \
  --remote-debugging-address=0.0.0.0 --remote-debugging-port=9222 \
  --user-data-dir=/home/node/.chromium-profile &
sleep 2

# Start VNC server (screen sharing)
x11vnc -display :99 -nopw -forever -shared -rfbport 5900 &

# Start NoVNC web viewer (so you can see the browser in your web browser)
/usr/share/novnc/utils/novnc_proxy --vnc localhost:5900 --listen 6080 &

# Start the OpenClaw gateway (the AI brain - this is the main process)
exec node dist/index.js gateway --bind "${OPENCLAW_GATEWAY_BIND:-lan}" --port 18789
```

Save: **Ctrl+O** → **Enter** → **Ctrl+X**

Make it executable:

```
chmod +x ~/openclaw-setup/unified-entrypoint.sh
```

Step 4.2: Simplify docker-compose.yml

First, back up the original:

```
cp ~/openclaw-setup/docker-compose.yml ~/openclaw-setup/docker-compose.yml.backup
```

Now edit it:

```
nano ~/openclaw-setup/docker-compose.yml
```

Delete everything in the file (**Ctrl+K** deletes one line at a time, keep pressing until empty).

Then type this:

```
services:
  openclaw:
    image: ${OPENCLAW_IMAGE:-openclaw:local}
    container_name: openclaw
    environment:
      HOME: /home/node
      TERM: xterm-256color
      DISPLAY: ":99"
      OPENCLAW_GATEWAY_TOKEN: ${OPENCLAW_GATEWAY_TOKEN}
    volumes:
      - ${OPENCLAW_CONFIG_DIR}:/home/node/.openclaw
      - ${OPENCLAW_WORKSPACE_DIR}:/home/node/.openclaw/workspace
      - openclaw-home:/home/node
      - ./unified-entrypoint.sh:/app/unified-entrypoint.sh
    ports:
      - "127.0.0.1:18789:18789"
      - "127.0.0.1:18790:18790"
      - "127.0.0.1:6080:6080"
    init: true
    restart: unless-stopped
    entrypoint: ["/bin/bash", "/app/unified-entrypoint.sh"]

volumes:
  openclaw-home:
```

Save: **Ctrl+O** → **Enter** → **Ctrl+X**

What did we change?

- Combined gateway + browser + CLI into ONE service called `openclaw`
- **All ports bound to 127.0.0.1** — only YOUR computer can access them. Nobody on your network can reach these ports.

- Added a named volume `openclaw-home` so browser logins and cookies persist through restarts
- Used our unified startup script that starts everything in order
- Set `restart: unless-stopped` so it auto-restarts after reboots

Understanding the ports:

Port	What It Does	Who Can Access It
<code>127.0.0.1:18789</code>	OpenClaw Gateway (the AI brain + web dashboard)	Only you (localhost)
<code>127.0.0.1:18790</code>	OpenClaw Bridge (device connections)	Only you (localhost)
<code>127.0.0.1:6080</code>	NoVNC (browser viewer)	Only you (localhost)

These are the only ports OpenClaw needs. They are **all bound to localhost** so they never touch the network. Your firewall doesn't even need to know about them.

Step 4.3: Delete the Override File (if it exists)

```
rm -f ~/openclaw-setup/docker-compose.override.yml
```

Step 4.4: Update the Browser Config

Tell OpenClaw to use the browser that's inside the same container (localhost):

```
nano ~/.openclaw/openclaw.json
```

Find the `"browser"` section (or add one if it doesn't exist). It should look like this:

```
"browser": {  
  "enabled": true,  
  "cdpUrl": "http://127.0.0.1:9222"  
}
```

If there was an old `cdpUrl` pointing to something like `openclaw-browser:9222` or `ws://...`, change it to `http://127.0.0.1:9222`.

Save: **Ctrl+O** → **Enter** → **Ctrl+X**

Step 4.5: Start the Unified Container

```
cd ~/openclaw-setup
docker compose down
docker compose up -d
```

Step 4.6: Verify ONE Container is Running

```
docker compose ps
```

You should see **ONE** container named `openclaw` with status "Up". Not two, not three. ONE.

PHASE 5: Set Up the Telegram Bot

Step 5.1: Create the Bot

1. Open **Telegram** on your phone
2. Search for **@BotFather** (it has a blue checkmark)
3. Send it the message: `/newbot`
4. It asks for a name — type something like "My AI Agent"
5. It asks for a username — type something like `my_ai_agent_bot` (must end in `bot`)
6. It gives you a **bot token** — a long string of numbers and letters. **SAVE THIS.**

Item	Your Value
Telegram Bot Token	_____
Telegram Bot Username	@_____

Step 5.2: Configure Telegram in OpenClaw

If you set this up during the onboarding wizard, it's already done. If not, you can configure it by editing the config file:

```
nano ~/.openclaw/openclaw.json
```

Add or update the Telegram section:

```
"channels": {  
  "telegram": {  
    "enabled": true,  
    "token": "YOUR_TELEGRAM_BOT_TOKEN_HERE",  
    "dmPolicy": "pairing"  
  }  
}
```

Replace `YOUR_TELEGRAM_BOT_TOKEN_HERE` with the token from Step 5.1.

Save: **Ctrl+O** → **Enter** → **Ctrl+X**

Then restart the container:

```
cd ~/openclaw-setup  
docker compose restart openclaw
```

PHASE 6: Start the Bot and Pair Your Account

Step 6.1: Check the Container is Running

```
cd ~/openclaw-setup  
docker compose logs openclaw --tail 10
```

You should see lines like:

- [heartbeat] started
- [gateway] listening on ws://0.0.0.0:18789
- [telegram] starting provider

If you see these, your agent is alive and running.

Step 6.2: Pair Your Telegram Account

Open Telegram on your phone and send your bot a message: "hello"

The bot will reply with a **pairing code** — a short code like `KEQ3UFNX` .

Now go to your terminal and approve yourself:

```
cd ~/openclaw-setup
docker exec openclaw node dist/index.js pairing approve telegram YOUR_CODE_HERE
```

Replace `YOUR_CODE_HERE` with the code the bot gave you (like `KEQ3UFNX`).

What just happened? You told OpenClaw "this Telegram account is mine, it's allowed to talk to the agent." From now on, ONLY your Telegram account can talk to the bot.

PHASE 7: Give the Agent a Brain (Connect Claude AI)

Without this step, the agent has no intelligence. It can't think or respond. This connects it to Claude.

Step 7.1: Generate a Setup Token

In a **new terminal** on your computer, run:

```
claude setup-token
```

This generates a long token that starts with `sk-ant-oat01-...`. **Copy the ENTIRE token.** Don't miss any characters.

Don't have the Claude CLI? Install it first:

```
npm install -g @anthropic-ai/claude-code
```

Or if you're using an Anthropic API key directly, you can skip this step and configure the API key in OpenClaw's config file instead.

Step 7.2: Register the Token with OpenClaw

```
cd ~/openclaw-setup
docker exec -it openclaw node dist/index.js models auth paste-token --provider anthropic
```

It will ask you to paste the token.

IMPORTANT: Long tokens often get cut off when pasting in the terminal. If your terminal has a **"Sanitize and Paste"** option (right-click and look for it), USE THAT. Regular paste can break long strings by adding invisible characters or line breaks.

Step 7.3: Restart the Agent

```
docker compose restart openclaw
```

Step 7.4: Verify the Token Works

```
docker exec openclaw node dist/index.js models status
```

You should see:

- Providers w/ OAuth/tokens (1): anthropic
- Token status showing static

Step 7.5: TEST IT!

Open Telegram and send your bot: "hello, how are you?"

It should respond with an actual, intelligent AI message. If it does — **your agent is alive and working!**

If you get "HTTP 401: Invalid bearer token": The token didn't register properly. Go back to Step 7.2 and try again. Make sure you paste the COMPLETE token using "Sanitize and Paste."

PHASE 8: Log Into Your Social Accounts Through the Browser

This is how your agent gets access to post on your behalf. You log in once, the session stays saved, and the agent uses it going forward.

Step 8.1: View the Remote Browser

Since everything is on localhost, you don't need an SSH tunnel. Just open your web browser and go to:

```
http://127.0.0.1:6080
```

You'll see a page that says "noVNC" with a **Connect** button. Click it.

Now you're looking at the browser running inside your container. You can click, type, and interact with it using your mouse and keyboard.

Step 8.2: Log Into Your Accounts

Navigate to the websites you want your agent to access (like x.com, instagram.com, etc.) and log in normally. Handle any 2FA, CAPTCHAs, or verification emails yourself.

Step 8.3: Done

The login sessions (cookies) are saved inside the container. They persist through restarts because we set up a persistent volume.

If a site logs you out later (expired session, security check), just open `http://127.0.0.1:6080` again and re-login.

PHASE 9: Access the Web Dashboard (Optional)

The dashboard is a web page where you can chat with the agent, manage settings, and see what it's doing. You don't NEED it — Telegram works fine. But it's nice for admin stuff.

Open your browser and go to:

```
http://127.0.0.1:18789/?token=YOUR_GATEWAY_TOKEN
```

Replace `YOUR_GATEWAY_TOKEN` with the token from Phase 3 Step 3.4.

If you forgot the token, get it from the terminal:

```
docker exec openclaw node dist/index.js dashboard --no-open
```

PHASE 10: Set Up Auto-Start on Boot

We want your agent to start automatically when your computer turns on.

Linux: Docker Handles It

We already set `restart: unless-stopped` in our `docker-compose.yml`, and Docker starts on boot. So your container will automatically restart when your computer restarts.

Verify:

```
sudo systemctl is-enabled docker
```

Should say `enabled` . If not:

```
sudo systemctl enable docker
```

macOS: Docker Desktop Handles It

Open **Docker Desktop** → **Settings** → **General** and check "**Start Docker Desktop when you log in**". Your container will auto-start because we set `restart: unless-stopped` .

PHASE 11: Set Up Scheduled Tasks (Cron Jobs)

OpenClaw has built-in scheduling. No extra software needed. Jobs are saved automatically and survive restarts.

Example: Schedule a Daily Task

```
docker exec openclaw node dist/index.js cron add \
  --name "Daily video post" \
  --cron "0 9 * * *" \
  --tz "America/New_York" \
  --session isolated \
  --message "Create and post today's viral video"
```

What the cron format means: `0 9 * * *` = "At minute 0 of hour 9, every day" = 9:00 AM every day.

Managing Scheduled Tasks

```
# See all scheduled tasks:
docker exec openclaw node dist/index.js cron list
```

```
# Run a task immediately:
docker exec openclaw node dist/index.js cron run JOB_ID

# Delete a task:
docker exec openclaw node dist/index.js cron remove JOB_ID
```

Terminal Tips for Beginners

"Nothing Happened" = It Worked

When you run a command and it just goes to a new line with no message, that means it **succeeded**. Linux only talks to you when something goes **WRONG**. Silence = success.

Copy and Paste in the Terminal

- **Ctrl+V** does **NOT** paste in most terminals (that's a Windows thing)
- Use **Ctrl+Shift+V** to paste
- Or **right-click** → **Paste**
- For long strings (like tokens), use "**Sanitize and Paste**" if your terminal offers it

Kitty Terminal Users

If you get this error: `Error opening terminal: xterm-kitty`

Run this before doing anything else:

```
export TERM=xterm
```

Nano Editor (The Text Editor)

- **Arrow keys** = move around
- **Ctrl+O** = save the file
- **Enter** = confirm the filename
- **Ctrl+X** = exit nano

- **Ctrl+K** = delete an entire line

Troubleshooting

Container Not Running

```
cd ~/openclaw-setup
docker compose ps
```

If the table is empty or shows "Exited":

```
docker compose up -d
```

Check what went wrong:

```
docker compose logs openclaw --tail 30
```

Telegram Bot Not Responding

1. Check if the container is running: `docker compose ps`
2. Check the logs: `docker compose logs openclaw --tail 20`
3. Make sure you paired your Telegram account (Phase 6)
4. Make sure the Claude token is registered (Phase 7)

"HTTP 401: Invalid Bearer Token"

Your Claude token stopped working. Re-register it:

1. Run: `claude setup-token`
2. Run:

```
docker exec -it openclaw node dist/index.js models auth paste-token --
provider anthropic
```
3. Paste the token using "Sanitize and Paste"
4. Restart: `docker compose restart openclaw`

"Error Opening Terminal: xterm-kitty"

```
export TERM=xterm
```

Token Gets Cut Off When Pasting

Make your terminal window **full screen** or as wide as possible. Long strings wrap at narrow widths and break. Always use "Sanitize and Paste" if your terminal offers it.

Firewall Blocking Something?

Check your firewall status:

```
sudo ufw status verbose
```

Remember: OpenClaw ports are all on localhost (127.0.0.1), so they bypass the firewall. You should NOT need to add any UFW rules for OpenClaw to work.

If you need to allow SSH from your local network for some reason:

```
sudo ufw allow from 192.168.0.0/16 to any port 22
```

Security Summary

```
YOUR LOCAL MACHINE
  |
  UFW Firewall (deny all incoming)
  |
  Outgoing allowed (APIs, Telegram, internet)
  |
  ONE DOCKER CONTAINER (sealed box)
  |
  OpenClaw + Chromium + FFMPEG + VNC (all inside)
  |
  All ports bound to 127.0.0.1 (localhost only)
```

```
|
Gateway token required for dashboard access
|
Telegram bot (paired to YOU only)
|
Auto-restart on reboot
```

Who can access your agent?

- YOU through localhost (127.0.0.1) — YES
- YOU through Telegram bot — YES
- Someone on your WiFi — NO (all ports are localhost-only + firewall blocks incoming)
- Random hackers — NO (firewall denies all incoming connections)

Key differences from the VPS guide:

VPS Guide	Local Guide
Needs Tailscale for remote access	No Tailscale needed — you're already at the machine
Needs SSH hardening	No SSH needed — you're already logged in
Creates separate <code>openclaw</code> user	Uses your existing user account
Removes root SSH access	Root user stays (it's your machine)
IONOS external firewall	Just UFW (or macOS firewall)
Needs SSH tunnel for NoVNC/ Dashboard	Direct access via localhost

Quick Reference Card

What	Command
Start the agent	<code>cd ~/openclaw-setup && docker compose up -d</code>
Stop the agent	<code>cd ~/openclaw-setup && docker compose stop</code>
	<code>cd ~/openclaw-setup && docker compose restart</code>

What	Command
Restart the agent	
View logs	<code>cd ~/openclaw-setup && docker compose logs openclaw --tail 20</code>
Check status	<code>cd ~/openclaw-setup && docker compose ps</code>
Check AI token	<code>docker exec openclaw node dist/index.js models status</code>
Pair Telegram	<code>docker exec openclaw node dist/index.js pairing approve telegram CODE</code>
List cron jobs	<code>docker exec openclaw node dist/index.js cron list</code>
View browser	Open <code>http://127.0.0.1:6080</code> in your browser
Open dashboard	Open <code>http://127.0.0.1:18789/?token=TOKEN</code> in your browser
Firewall status	<code>sudo ufw status verbose</code>
Disk space	<code>df -h</code>
Memory usage	<code>free -h</code>

Fill In Your Values

Keep this filled in for quick reference:

Item	Your Value
Gateway Token	_____
Telegram Bot Token	_____
Telegram Bot Username	@_____

Sources

- [OpenClaw GitHub](#)
- [OpenClaw Documentation](#)
- [OpenClaw Docker Setup](#)
- [OpenClaw Security](#)
- [OpenClaw Cron Jobs](#)